Machine Learning for Business

GTSC2143

Lecture 2 Data Processing Using Python

Instructor: Dr. Lily Bei Luo

AY2025-2026 Semester 1

北京师范大学 香港浸会大学 联合国际学院
BEIJING NORMAL UNIVERSITY · HONG KONG BAPTIST UNIVERSITY
UNITED INTERNATIONAL COLLEGE

Section 1 Pandas

Pandas

- Pandas, like NumPy, is one of the most popular Python libraries for data analysis.
- It is a high-level abstraction over low-level NumPy, which is written in pure C.
- Pandas provides high-performance, easy-to-use data structures and data analysis tools.
- There are two main structures used by pandas; *data frames* and *series*.

	height	Weight	Weight2	age	Gender
Amy	160	125	126	32	2
Bob	170	167	155	-1	1
Chris	168	143	150	28	1
David	190	182	NA	42	1
Ella	175	133	138	23	2
Frank	172	150	148	45	1

	salary	Credit score
Alice	50000	700
Bob	NA	670
Chris	60000	NA
David	-99999	750
Ella	70000	685
Tom	45000	660

Series and DataFrame

Pandas provides a couple of very useful datatypes:

- Series represents 1D data, like time series, calendars, the output of one-variable functions, etc.
- DataFrame represents 2D data, like a column-separated-values (CSV) file, a microarray, a database table, a matrix, etc.

Each column of a DataFrame is a Series.

- That's why we will see how the Series data type works first.
- Most of what we will say about Series also applies to DataFrames.

Pandas

Series

A Series is a one-dimensional array with a labeled axis, that can hold arbitrary objects.

The axis is called the index, and can be used to access the elements; it is very flexible, and not necessarily numerical.

It works partially like a list and partially like a dict.

Creating a Series

It is possible to specify just the series data, associating an implicit numeric index.

```
import pandas as pd
s = pd.Series(["a", "b", "c"])
print(s)
```

```
>> 0 a
   1 b
   2 c
```

Accessing a Series

Let's create a Series representing the hours of sleep we had the chance to get each day of the past week. We may now access it through either the position (as a list) or the index (as a dict)

```
import pandas as pd

days = ["mon", "tue", "wed", "thu", "fri"]

sleephours = [6, 2, 8, 5, 9]

s = pd.Series(sleephours, index=days)

print(s["mon"])
```

```
>> 6
```

```
s["tue"]=3

print(s[1])
```

```
>> 3
```

Slicing a Series

We can also slice the positions, like we would do with a list. Note that both the data and the index are extracted correctly. It also works with labels.

```
print(s[-3:])

print(s["tue":"thu"])
```

```
>> wed 8
   thu 5
   fri 9
```

```
>> tue 2
   wed 8
   thu 5
```

Head and tail

The first and last n elements can be extracted also using head() and tail().

```
print(s.head(2))
print(s.tail(3))
```

```
>> mon 6
   tue 2
```

```
>> wed 8
   thu 5
   fri 9
```

Operator broadcasting

The Series class automatically broadcasts arithmetical operations by a scalar to all of the elements.

```
print(s)
print(s+1)
print(s*2)
```

```
>> mon 6
   tue 2
   wed 8
   thu 5
   fri 9
```

```
>> mon 7
   tue 3
   wed 9
   thu 6
   fri 10
```

```
>> mon 12
   tue 4
   wed 16
   thu 10
   fri 18
```

Automatic label assignments

Operations between multiple time series are automatically aligned by label, meaning that elements with the same label are matched prior to carrying out the operation.

```
print(s[1:])
print(s[:-1])
print(s[1:]+s[:-1])
```

```
>> tue 2
   wed 8
   thu 5
   fri 9
```

```
>> mon 6
   tue 3
   wed 8
   Thu 5
```

```
>> fri NaN
   mon NaN
   thu 10
   tue 4
   wed 16
```

Not-a-Number (NaN)

The index of the resulting Series is the union of the indices of the operands. What happens depend on whether a given label appears in both input Series or not:

For common labels (in our case "tue", "wed", "thu"), the output Series contains the sum of the aligned elements.

For labels appearing in only one of the operands ("mon" and "fri"), the result is a NaN, i.e. not-a-number.

NaN is just a symbolic constant that specifies that the object is a number-like entity with an invalid or undefined value.

Dealing with missing values

There are different strategies for dealing with nan‘s. There is no “best” strategy: you have to pick one depending on the problem you are trying to solve.

```
t = s[1:]+s[:-1]
print(t)
print(t.dropna())
print(t.fillna(0))
```

```
>> fri NaN
   mon NaN
   thu 10
   tue 4
   wed 16
```

```
>> thu 10
   tue 4
   wed 16
```

```
>> fri 0
   mon 0
   thu 10
   tue 4
   wed 16
```

Pandas

Computing statistics

```
print(s)
```

```
>> mon 6
   tue 2
   wed 8
   thu 5
   fri 9
```

```
print(s.sum())
print(s.prod())
print(s.max())
print(s.argmax())
print(s.mean())
print(s.var())
print(s.std())
print(s.median())
print(s.cumsum())
```

```
>> 30
>> 4320
>> 9
>> fri
>> 6.0
>> 7.5
>> 2.7386127875
>> 6.0
>> mon 6
   tue 8
   wed 16
   thu 21
   fri 30
```

Computing statistics

A quick way to get several useful statistics is to use s.describe(). Anyway, the list of statistical methods associated with Series is larger than this.

```
print(s.describe())
```

```
>> count  5.0000
   mean   6.0000
   Std    2.7386
   min    2.0000
   25%    5.0000
   50%    6.0000
   75%    8.0000
   max    9.0000
```

DataFrame

Pandas DataFrame is the 2D analogue of a Series: it is essentially a table of heterogeneous objects.

A DataFrame holds three major attributes:

- the index, which holds the labels of the rows
- the columns, which holds the labels of the columns
- the shape, which describes the dimension of the table

When you extract a column from a DataFrame you get a proper Series, and you can operate on it using all the tools presented in the previous section.

Further, most (not all) of the operations that you can do on a Series, you can also do on an entire DataFrame.

Creating a DataFrame

```
import pandas as pd
d = {"name": pd.Series(["bobby","ronald","ronald","ronald"]),
   "surname": pd.Series(["fisher","fisher","reagan","mcdonald"])}
df = pd.DataFrame(d)
print(df)
```

```
>>  name    surname
    1   bobby   fisher
    2   ronald  fisher
    3   ronald  fisher
    4   ronald  reagan
```

- The keys of the dictionary became the columns of the dataframe
- The index of the various Series became the index of the dataframe

Creating a DataFrame

```
import pandas as pd
d = {"column1": [1.,2.,6.,-1.],
     "column2": [0.,1.,-2.,4.]}
df = pd.DataFrame(d)
print(df)
```

```
>>  column1   column2
    0   1.0     0.0
    1   2.0     1.0
    2   6.0     -2.0
    3   -1.0    4.0
```

- The columns are taken from the keys
- The index is set to the default one

Creating a DataFrame

```
import pandas as pd
d = [ {"a": 1, "b": 2},
    {"a": 2, "c": 3}]
df = pd.DataFrame(d)
print(df)
```

```
>>  a   b   c
    0   1   2   NaN
    1   2   NaN 3
```

- The columns are taken from the keys of the dictionaries
- The index is the default one.
- Since not all common keys appear in all input dictionaries, missing values (i.e. NaN's) are automatically added.

Pandas

Loading a CSV file

```
import pandas as pd

df = pd.read_csv("iris.csv")

print(df.head())
```

```
   SepalLength SepalWidth PetalLength  PetalWidth          Name
0          5.1        3.5          1.4         0.2  Iris-setosa
1          4.9        3.0          1.4         0.2  Iris-setosa
2          4.7        3.2          1.3         0.2  Iris-setosa
3          4.6        3.1          1.5         0.2  Iris-setosa
4          5.0        3.6          1.4         0.2  Iris-setosa
5          5.4        3.9          1.7         0.4  Iris-setosa
```

Pandas

Extracting rows and columns

Operation	Syntax	Result
Select column	df[col]	Series
Select multiple columns	df[[col1,col2]]	DataFrame
Select row by label	df.loc[label]	Series
Select row by integer location	df.iloc[loc]	Series
Slice rows	df[5:10]	DataFrame
Select rows by boolean vector	df[bool_vec]	DataFrame

Pandas

Masking and Filtering

```
print(df["PetalLength"][df.PetalLength > 5])

print(df["Name"][df.Name == "Iris-virginica"])

print(df[["Name","PetalLength","SepalLength‟]][
    df.Name == "Iris-virginica"]])
```

```
>> 83      5.1
   100     6.0

>> 100     Iris-virginica
   101     Iris-virginic

>> Name           PetalLength   SepalLength
   100 Iris-virginica   6.0     6.3
   101 Iris-virginica   5.1     5.8
```

Pandas

Sorting

```
df_sorted = df.sort_values(by ='PetalLength’)
```

```
df_sorted = df.sort_values(by =['PetalLength', 'PetalWidth’],
              ascending = [True, False])
```

Statistics

Statistics can be computed on rows, columns, or the whole table

```
print(df.loc[114][:-1].mean()) # Exclude last column, Name
print(df.PetalLength.mean())
print(df.mean())
```

```
>> 4.025
>> 3.76
>> SepalLength   5.84
   SepalWidth    3.05
   PetalLength   3.76
   PetalWidth    1.20
```

Section 2 Data Processing

Merge

Merging different dataframes is performed using the merge() function.

Merging means that, given two tables with a common column name, first the rows with the same column value are matched; then a new table is created by concatenating the matching rows.

```
sequences = pd.DataFrame({
"id":  ["Q99697",  "O18400",  "P78337",  "Q9W5Z2"],
"seq": ["METNCR", "MDRSSA", "MDAFKG", "MTSMKD"],
})

names = pd.DataFrame({
"id":  ["Q99697",  "O18400",  "P78337",  "P59583"],
"name": ["PITX2_HUMAN", "PITX_DROME",
"PITX1_HUMAN", "WRK32_ARATH"],
})
```

Data Processing

Merge – Inner

```
print(sequences)
print(names)
print(pd.merge(sequences,  names,  on="id",  how="inner"))
```

```
       id     seq
0   Q99697   METNCR
1   O18400   MDRSSA
2   P78337   MDAFKG
3   Q9W5Z2   MTSMKD
```

```
       id            name
0   Q99697     PITX2_HUMAN
1   O18400      PITX_DROME
2   P78337     PITX1_HUMAN
3   P59583     WRK32_ARATH
```

```
       id     seq     name
0   Q99697   METNCR PITX2_HUMAN
1   O18400   MDRSSA  PITX_DROME
2   P78337   MDAFKG PITX1_HUMAN
```

Mismatched ids are dropped!

Data Processing

Merge – Left

```
print(sequences)
print(names)
print(pd.merge(sequences,  names,  on="id",  how="left"))
```

```
       id     seq                    id     name
0  Q99697  METNCR                0  Q99697  PITX2_HUMAN
1  O18400  MDRSSA                1  O18400   PITX_DROME
2  P78337  MDAFKG                2  P78337  PITX1_HUMAN
3  Q9W5Z2  MTSMKD                3  P59583  WRK32_ARATH

       id     seq      name
0  Q99697  METNCR  PITX2_HUMAN
1  O18400  MDRSSA   PITX_DROME
2  P78337  MDAFKG  PITX1_HUMAN
3  Q9W5Z2  MTSMKD         NaN
```

The ids are taken from the left table!

Data Processing

Merge – Right

```
print(sequences)
print(names)
print(pd.merge(sequences,  names,  on="id",  how="right"))
```

```
        id      seq                    id      name
0   Q99697   METNCR                0  Q99697 PITX2_HUMAN
1   O18400   MDRSSA                1  O18400  PITX_DROME
2   P78337   MDAFKG                2  P78337 PITX1_HUMAN
3   Q9W5Z2   MTSMKD                3  P59583 WRK32_ARATH

        id      seq      name
0   Q99697   METNCR PITX2_HUMAN
1   O18400   MDRSSA  PITX_DROME
2   P78337   MDAFKG PITX1_HUMAN
3   P59583      NaN WRK32_ARATH
```

The ids are taken from the right table!

Data Processing

Merge – Outer

```
print(sequences)
print(names)
print(pd.merge(sequences,  names,  on="id",  how="outer"))
```

```
        id      seq
0   Q99697   METNCR
1   O18400   MDRSSA
2   P78337   MDAFKG
3   Q9W5Z2   MTSMKD
```

```
        id          name
0   Q99697   PITX2_HUMAN
1   O18400    PITX_DROME
2   P78337   PITX1_HUMAN
3   P59583   WRK32_ARATH
```

```
        id      seq     name
0   Q99697   METNCR PITX2_HUMAN
1   O18400   MDRSSA  PITX_DROME
2 P78337      MDAFKG PITX1_HUMAN
3 Q9W5Z2      MTSMKD      NaN
4 P59583         NaN WRK32_ARATH
```

All ids are retained!

Grouping Tables

The ***groupby*** method is essential for efficiently performing operations on groups of rows.

Given the Iris dataset, we want to compute the average of the four columns for each of the three different Iris species.

The result should be a dataframe with 3 species (rows) by 4 columns (petal/sepal length/width).

Grouping Tables

Iterating over the grouped variable returns (value-of-Name, DataFrame) tuples:

- The 1st item is the value of the column Name shared by the group
- The 2nd item is a DataFrame including only the rows in that group.

```
grouped = iris.groupby(iris.Name)
for group in grouped:
    print(group[0], group[1].shape)
```

```
>> Iris-setosa (50, 5)
   Iris-versicolor (50, 5)
   Iris-virginica (50, 5)
```

Data Processing

Grouping Tables

Similar to dplyr() function in R

```
print (grouped.mean())
print (grouped["SepalLength"].mean())
```

Name	SepalLength	SepalWidth	PetalLength	PetalWidth
Iris-setosa	5.006	3.418	1.464	0.244
Iris-versicolor	5.936	2.770	4.260	1.326
Iris-virginica	6.588	2.974	5.552	2.026

Name	SepalLength
Iris-setosa	5.006
Iris-versicolor	5.936
Iris-virginica	6.588

Grouping Tables

It is possible to apply some transformation (e.g. mean()) to the individual groups automatically, using the aggregate() method directly on the grouped variable. The result of aggregate() is a dataframe.

```
iris_mean_by_name = grouped.agg(['mean', 'sum', 'count'])
print(iris_mean_by_name)
```

	SepalLength	SepalWidth	PetalLength	PetalWidth
Name				
Iris-setosa	5.006	3.418	1.464	0.244
Iris-versicolor	5.936	2.770	4.260	1.326
Iris-virginica	6.588	2.974	5.552	2.026

Applying Custom Functions

There will be times when you want to apply a function that is not built-in to Pandas. df.apply() applies a function column-wise or row-wise across a dataframe.

```
print(iris[['SepalLength','SepalWidth']].apply(np.sin))
```

```
>>  SepalLength   SepalWidth
    0   -0.93      -0.35
    1  -0.98       0.14
    2  -1.00       -0.06
    3  -0.99       0.04
```

Tutorial